

AI-DRIVEN SEMANTIC SEARCH AND RETRIEVAL SYSTEM

A Project Report Submitted in partial fulfillment for the award of
Degree of Bachelor of Technology
In
Computer Science & Information Technology

Submitted By

**Love Kumar
(240089020062)**

**Navneet Patel
(240089020066)**

**Charu Patel
(240089020031)**

**Garima Saxena
(240089020033)**

Under Supervision of

Dr. Pooja Yadav



Department of Computer Science & Information Technology

Faculty of Engineering & Technology

M.J.P. Rohilkhand University, Bareilly

Acknowledgement

We are extremely grateful and remain indebted to our mentor, **Dr. Pooja Yadav**, for being a source of inspiration and for her constant support in the Designing, Implementation, and Evaluation of the project. We are thankful to her for her constant constructive criticism and valuable suggestions, which lead us to a beneficial stage while developing the project on “**AI-DRIVEN SEMANTIC SEARCH AND RETRIEVAL SYSTEM**”.

She has been a constant source of inspiration and motivation for hard work. She has been very cooperative throughout this project work. Through this column, it would be our utmost pleasure to express our warm thanks to her for her encouragement, co-operation, and consent without which we might not be able to accomplish this project.

Love Kumar (240089020062)

Navneet Patel (240089020066)

Charu Patel (240089020031)

Garima Saxena (240089020033)

Candidate's Declaration

I hereby declared that project report titled “**AI-DRIVEN SEMANTIC SEARCH AND RETRIEVAL SYSTEM**” is prepared by me based on available literature and I have not submitted it anywhere else for the award of any other degree or diploma.

Date:

Love Kumar (240089020062)

Navneet Patel (240089020066)

Charu Patel (240089020031)

Garima Saxena (240089020033)

Certificate from Supervisor

I certify that the above statement made by the candidate is true to the best of my knowledge

Date:

Dr. Pooja Yadav

INDEX

Acknowledgement	i
Candidate's Declaration	ii
Certificate	iii
1. Introduction	1-4
2. Related Work	4-5
3. System Analysis and Design	5-10
4. Implementation	11-13
5. Results and Discussion	14-15
6. Conclusion	15
References	

1. Introduction

The rapid growth of Artificial Intelligence (AI), Large Language Models (LLMs), and semantic search systems has created a need for efficient storage and retrieval of high-dimensional vector embeddings. Traditional databases are optimized for exact keyword-based searches but struggle to handle semantic similarity queries efficiently. Vector databases have emerged as a specialized solution for storing embeddings generated by machine learning models and enabling fast nearest-neighbor search.

This project presents the design and implementation of an AI-powered Vector Database and Retrieval-Augmented Generation (RAG) system developed in C++. The system integrates multiple similarity search algorithms, including Brute Force Search, KD-Tree, and Hierarchical Navigable Small World (HNSW) graphs, to support efficient retrieval of semantically similar data. Additionally, the system incorporates Ollama-based embedding and language models to enable document indexing, semantic search, and AI-assisted question answering.

1.1 Background and Motivation

Recent advancements in Artificial Intelligence have significantly increased the use of vector embeddings for representing text, images, and other forms of data in a numerical format. Modern AI applications such as semantic search engines, recommendation systems, chatbots, and Retrieval-Augmented Generation (RAG) systems rely heavily on efficient similarity search over large collections of embeddings. Traditional databases are not designed for high-dimensional nearest-neighbor searches and often become inefficient when dealing with embedding vectors. To address this challenge, specialized vector indexing techniques such as KD-Trees and HNSW graphs have been developed. These methods enable faster retrieval while maintaining high search accuracy. The motivation behind this project is to understand and implement the core technologies behind modern vector databases and RAG systems. By integrating multiple indexing algorithms and local

language models, the project demonstrates how semantic retrieval and AI-powered question answering can be achieved within a unified system.

1.2 Problem Statement

As the volume of embedding-based data continues to grow, efficient storage and retrieval of high-dimensional vectors becomes increasingly challenging. Performing similarity search using traditional linear scanning methods results in high computational costs and poor scalability.

1.3 Objectives

The primary objectives of this project are:

1. To design and implement a vector database capable of storing high-dimensional embeddings.
2. To implement multiple similarity search techniques, including Brute Force Search, KD-Tree, and HNSW.
3. To evaluate and compare the performance of different search algorithms.
4. To support multiple distance metrics such as Euclidean, Cosine, and Manhattan distance.
5. To develop a document management system for storing and indexing textual data.
6. To generate embeddings using locally hosted AI models through Ollama.
7. To implement a Retrieval-Augmented Generation (RAG) pipeline for question answering.

1.4 Scope of the Project

The scope of this project includes the implementation of a complete vector database and document retrieval framework with AI integration. The system supports vector insertion, deletion, indexing, similarity search, and benchmarking of different nearest-neighbor algorithms. It also provides facilities for document chunking, embedding generation, semantic retrieval, and question answering using a locally hosted language model.

The project focuses on demonstrating the fundamental concepts and architecture of modern vector databases rather than competing with large-scale commercial solutions. It is intended for educational, research, and experimental purposes and can serve as a foundation for future enhancements such as distributed storage, hybrid search, metadata filtering, advanced ranking techniques, and cloud deployment.

1.5 Organization of the Project

This project report is organized into several Sections to provide a clear understanding of the system design, development, and implementation process.

- **Section 1: Introduction**

Provides background, motivation, problem definition, objectives, scope, and structure of the project.

- **Section 2: Related Work**

Discusses existing systems, research work, and technologies

- **Section 3: System Analysis and Design**

Explains system requirements, architecture, UML diagrams, and design methodologies.

- **Section 4: Implementation**

Describes the tools, technologies, algorithms, and coding used in developing the application.

- **Section 5: Results and Discussion**

Presents the output, performance analysis, and evaluation of the system.

- **Section 6: Conclusion**

Summarizes the project and discusses possible improvements and future enhancements.

2. Related Work

The increasing adoption of Artificial Intelligence (AI) and Large Language Models (LLMs) has led to the widespread use of vector databases for semantic search and information retrieval. Traditional database systems are primarily designed for exact keyword matching and structured queries, making them less suitable for handling high-dimensional embedding vectors generated by modern machine learning models [1], [2].

Several vector search techniques have been proposed to improve retrieval efficiency. Brute Force Search provides exact nearest-neighbor results but becomes computationally expensive as the dataset size increases. Tree-based indexing methods such as KD-Trees improve search performance by organizing vectors hierarchically, while graph-based approaches such as Hierarchical Navigable Small World (HNSW) graphs provide fast and scalable approximate nearest-neighbor search for high-dimensional data [1], [2].

Modern vector database platforms such as Pinecone, Weaviate, and Milvus employ advanced indexing techniques, including HNSW, to support large-scale semantic search applications [1]. Furthermore, Retrieval-Augmented Generation (RAG) has emerged as an effective technique for improving the quality and factual accuracy of LLM-generated responses by combining information retrieval with text generation [3], [4].

Recent research has explored several variants of RAG, including graph-based retrieval methods, hybrid retrieval architectures, and active retrieval mechanisms to improve retrieval quality and response relevance [5], [6], [7]. In addition, locally deployable AI frameworks such as Ollama have enabled organizations and researchers to run embedding and language models on their own infrastructure, reducing dependency on cloud-based services while maintaining privacy and cost efficiency [8].

Research Gap and Proposed Solution

Although several commercial and open-source vector database solutions are available, many of them operate as black-box systems, making it difficult for students and researchers to understand the internal implementation of vector indexing and retrieval algorithms [1], [2]. Moreover, most educational implementations focus on a single retrieval technique and do not provide a platform for comparing multiple indexing approaches under the same environment.

To address these gaps, the proposed system:

- Implements and compares Brute Force, KD-Tree, and HNSW search algorithms.
- Supports semantic search using vector embeddings and multiple distance metrics.
- Integrates document storage, retrieval, and Retrieval-Augmented Generation (RAG).
- Utilizes locally hosted Ollama models for embedding generation and AI-powered question answering.

3. System Design

3.1 Introduction

This chapter describes the design and architecture of the proposed AI-powered Vector Database and Retrieval-Augmented Generation (RAG) system. The system combines vector indexing algorithms, document storage, semantic search, and local language model integration to enable efficient retrieval and question-answering capabilities. The design follows a modular architecture, allowing different indexing and retrieval techniques to operate within a unified framework.

Refer to **Figure 3.1** for the overall system architecture and **Figure 3.2** for the user interaction flow.

3.2 System Requirements

3.2.1 Functional Requirements

The system shall:

- Store and manage vector embeddings.
- Support document insertion and deletion.
- Generate embeddings using Ollama models.
- Perform similarity search using Brute Force, KD-Tree, and HNSW.
- Support multiple distance metrics (Cosine, Euclidean, Manhattan).
- Retrieve relevant document chunks for a query.
- Generate answers using the RAG pipeline.
- Provide REST API endpoints for all operations.
- Display search results and benchmarking information.

3.2.2 Non-Functional Requirements

The system should:

- Provide fast retrieval for large datasets.
- Maintain scalability with increasing document volume.
- Ensure modular and maintainable code structure.
- Support local deployment without cloud dependency.
- Provide reliable API responses.
- Allow easy integration of additional retrieval algorithms in the future.

3.3 System Architecture

The system consists of five major components:

1. User Interface Layer
2. HTTP API Layer
3. Vector Database Engine
4. Document Database

5. Ollama-based Embedding and Language Models

When a user submits a document, the system chunks the text, generates embeddings, and stores them within the document database. During querying, the question is converted into an embedding, relevant document chunks are retrieved using the selected indexing algorithm, and the retrieved context is provided to the language model for answer generation.

The overall architecture of the system is illustrated in **Fig 3.1**.

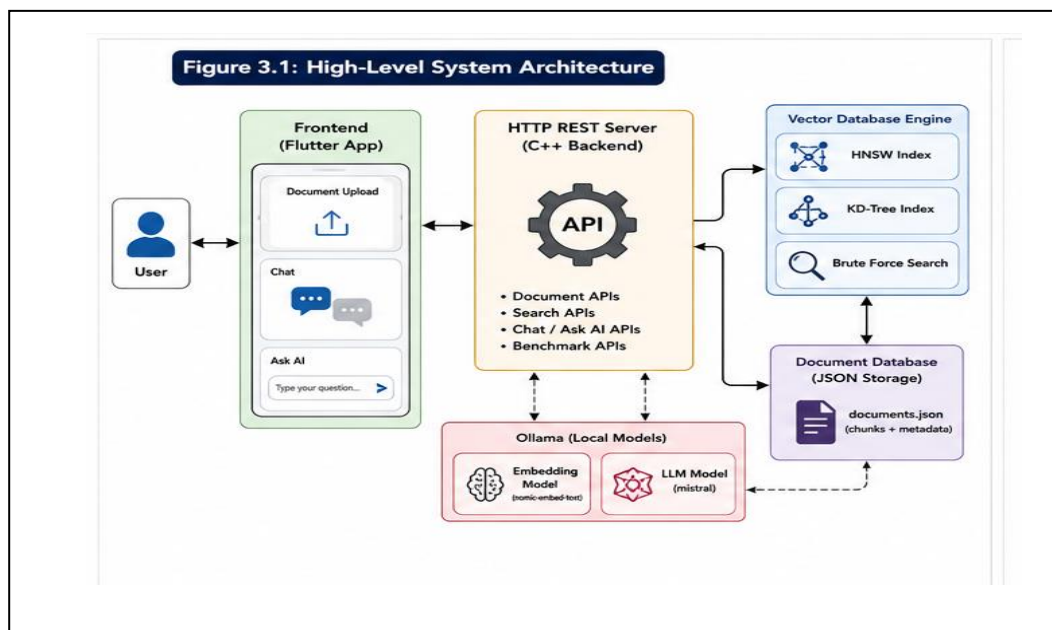


Fig 3.1 System Architecture

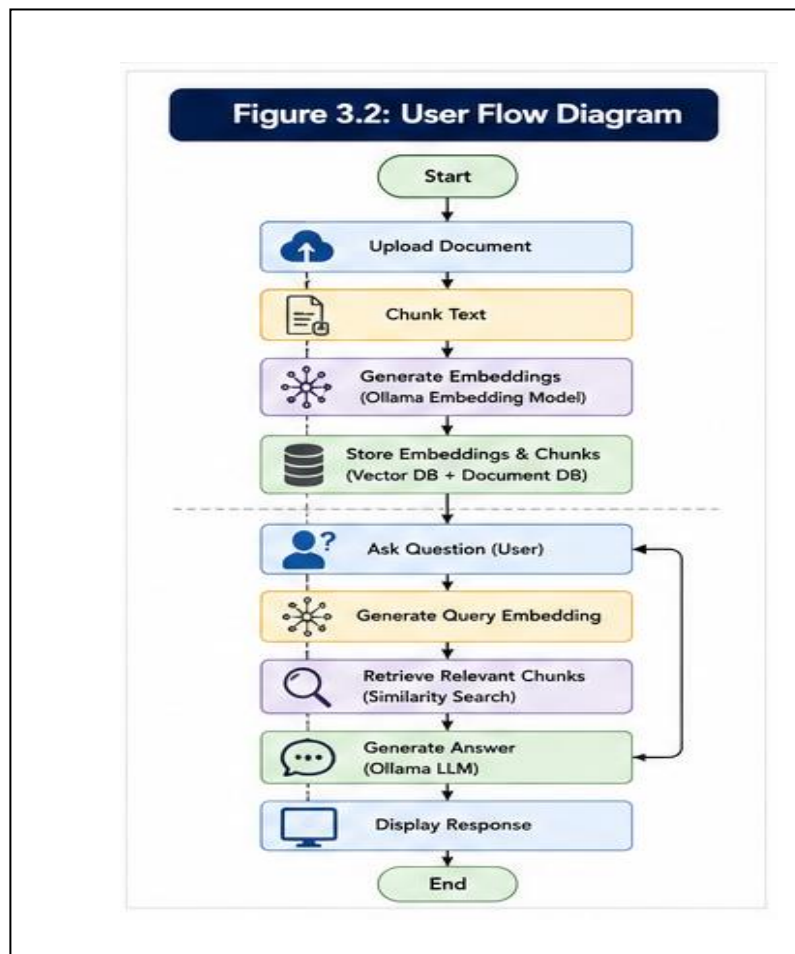


Fig 3.2 User Flow Diagram

3.4 UML Diagrams

UML diagrams provide a graphical representation of system interactions and workflow.

3.4.1 Use Case Diagram

The use case diagram shown in **Figure 3.3** illustrates the interactions between the user and the system, including document management, semantic search, benchmarking, and question answering.

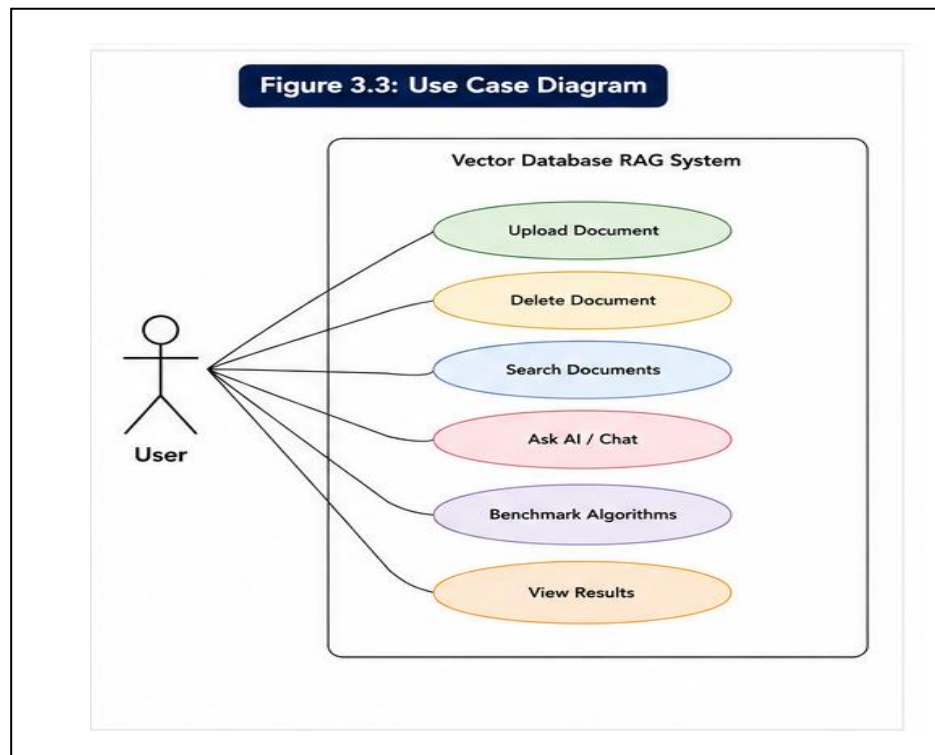


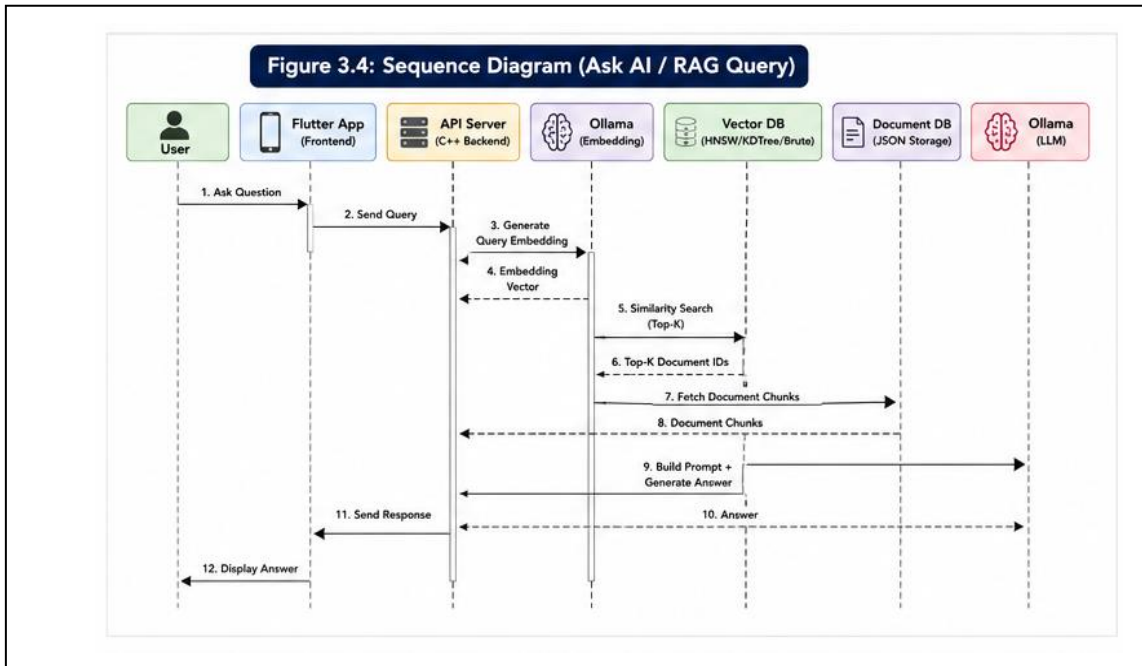
Fig 3.3 Use cases of the system

3.4.2 Sequence Diagram

The sequence diagram shown in **Figure 3.4** illustrates the execution flow of the Retrieval-Augmented Generation process, from user query submission to answer generation.

3.4.3 Activity Diagram / Flowchart

The activity diagram shown in **Figure 3.5** represents the document retrieval and RAG workflow implemented within the system



.Fig 3.4 Sequence Diagram

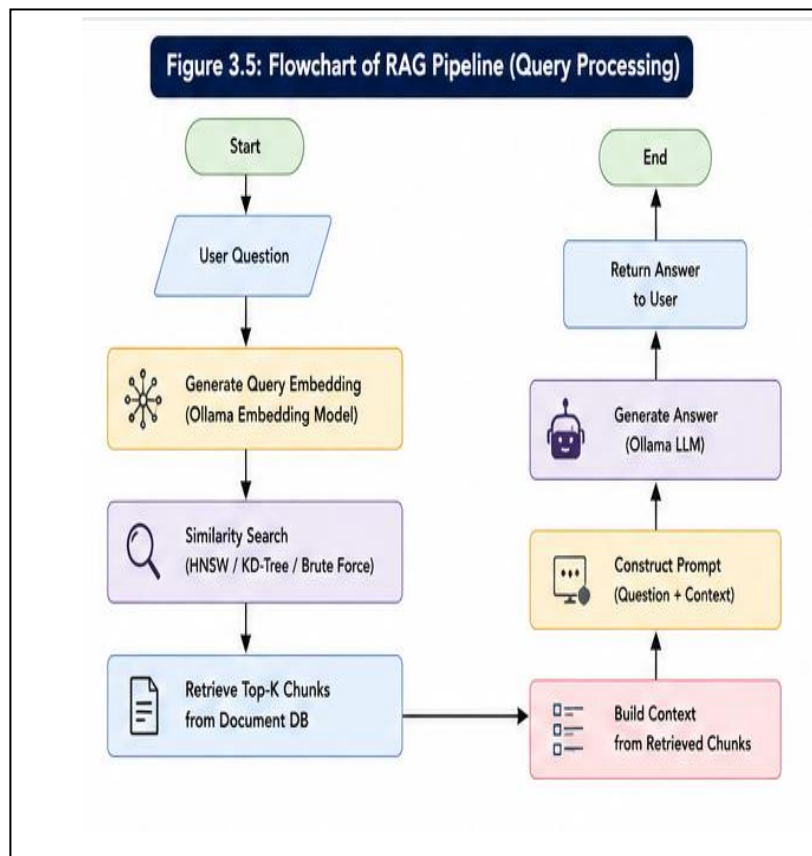


Fig 3.5 Flowchart showing the working of Rag

4. Implementation

This chapter describes the implementation of the proposed AI-Powered Vector Database and Retrieval-Augmented Generation (RAG) system. The system was developed using a C++ backend and a Flutter frontend. It integrates vector indexing algorithms, document management, semantic retrieval, and local AI models to provide efficient document search and question-answering capabilities.

The implementation follows the architecture presented in Figure 3.1.

4.1 Tools and Technologies Used

The technologies used during development are listed in Table 4.1.

Table 4.1: Tools and Technologies Used

Component	Technology Used	Purpose
Frontend	Flutter	User interface and interaction
Backend	C++	Core application logic
HTTP Server	cpp-http lib	REST API implementation
Vector Indexing	HNSW, KD-Tree, Brute Force	Similarity search
Embedding Model	nommic-embed-text	Text embedding generation
Language Model	llama3.2	Response generation
AI Platform	Ollama	Local model hosting
Storage	JSON Files	Document persistence
Development Environment	Visual Studio Code	Development and debugging

Table 4.1 summarizes the primary technologies used in the development of the system.

4.2 System Architecture Implementation.

The implementation consists of four major layers:

- Flutter Frontend Layer
- REST API Layer

3. Vector Database Layer
4. AI Processing Layer

The Flutter frontend allows users to upload documents and interact with the AI assistant. The backend exposes REST APIs for document management, retrieval, and question answering. The vector database layer handles indexing and similarity search using HNSW, KD-Tree, and Brute Force algorithms. The AI processing layer utilizes Ollama for embedding generation and response generation.

The implementation architecture corresponds to the design shown in Figure 3.1.

4.3 Module Implementation

The system is divided into the modules shown in Table 4.2.

Table 4.2: System Modules

Module	Description
Document Upload Module	Accepts and stores uploaded documents
Text Chunking Module	Splits documents into manageable chunks
Embedding Module	Generates vector embeddings using Ollama
Vector Indexing Module	Stores embeddings using HNSW, KD-Tree, and Brute Force
Retrieval Module	Performs similarity search and retrieves relevant chunks
RAG Module	Builds context and generates responses
Chat Module	Provides conversational interaction with users

Table 4.2 presents the functional modules implemented in the system.

4.4 Implementation Steps

The implementation was carried out in the following sequence:

1. Development of the backend REST API using C++.
2. Implementation of vector indexing algorithms (Brute Force, KD-Tree, and HNSW).
3. Integration of Ollama embedding and language models.
4. Development of document upload and storage functionality.
5. Implementation of text chunking and embedding generation.

6. Development of semantic retrieval mechanisms.
7. Construction of the Retrieval-Augmented Generation (RAG) pipeline.
8. Development of the Flutter-based user interface.
9. Integration of frontend and backend components.
10. Testing and validation of document retrieval and question-answering functionality.

The workflow followed during implementation is illustrated in Figure 4.1.

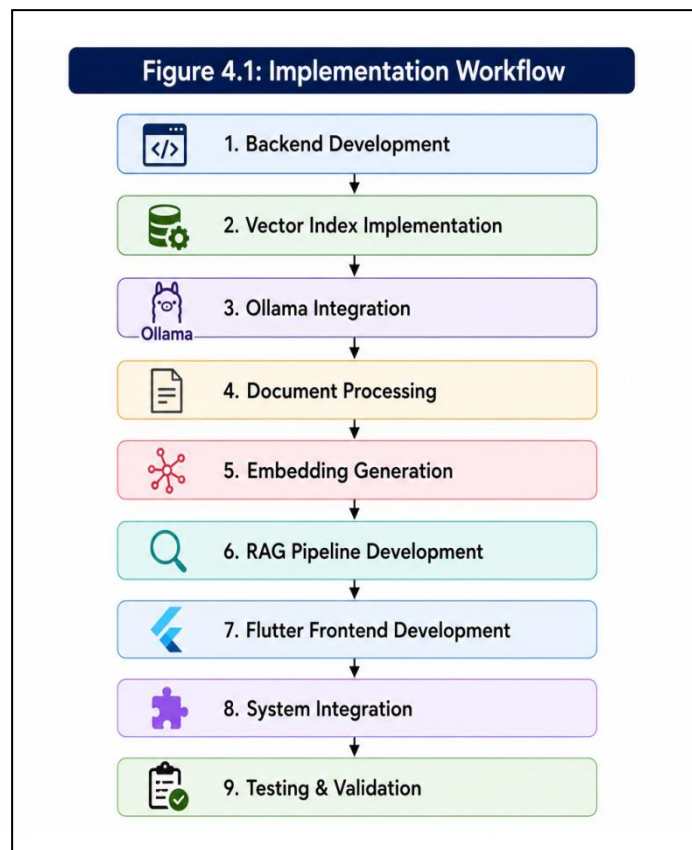


Fig 4.1 The workflow of the project

5. Results and Discussions

This chapter presents the results obtained from the implementation of the AI-Powered Vector Database and Retrieval-Augmented Generation (RAG) system. The results demonstrate the successful integration of document processing, vector indexing, semantic

retrieval, and AI-assisted question answering. The chapter also discusses the performance and functionality of the developed system.

5.1 Application Interface Results

The Flutter-based application provides an intuitive interface for document upload and AI interaction. Users can upload documents, store them within the vector database, and ask natural language questions through the chat interface.

The major interface of the application is shown in Figure 5.1

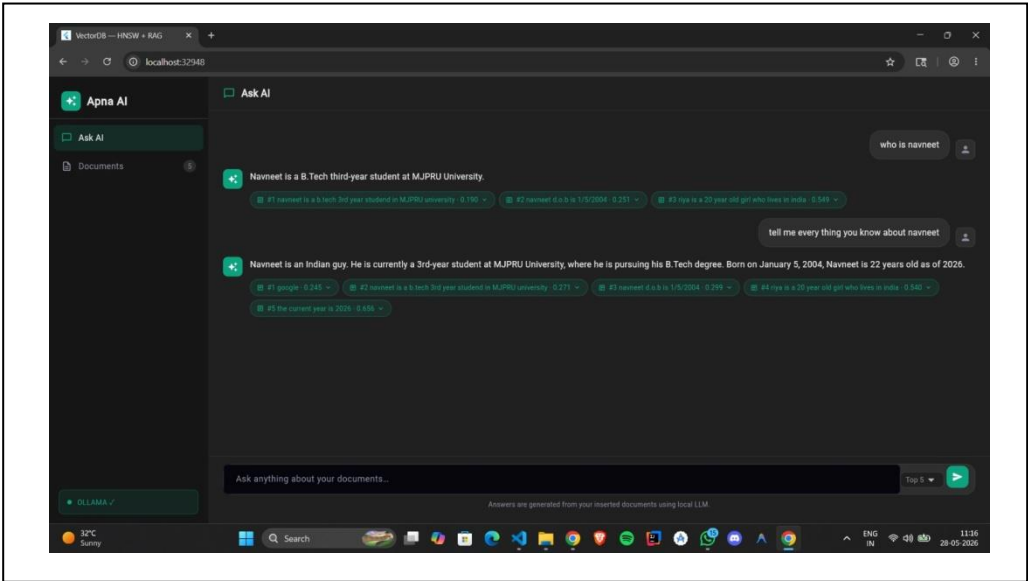


Fig 5.1: The chat interface

5.2 System Performance Analysis

The system was evaluated based on retrieval efficiency, response generation capability, and overall functionality.

Table 5.1: Functional Evaluation Results

Feature	Status
Document Upload	Successful
Text Chunking	Successful
Embedding Generation	Successful
Vector Storage	Successful

Similarity Search	Successful
Document Retrieval	Successful
RAG-based Question Answering	Successful
REST API Communication	Successful
Flutter Integration	Successful

Table 5.1 shows that all major functionalities were implemented and tested successfully.

Table 5.2: Implemented Search Algorithms

Algorithm	Purpose
Brute Force	Exact similarity search
KD-Tree	Hierarchical vector search
HNSW	Approximate nearest-neighbor search

Among the implemented approaches, HNSW provided the most efficient retrieval for larger datasets, while Brute Force produced exact results at the cost of increased search time. KD-Tree offered a balance between search accuracy and retrieval speed for moderate-sized datasets.

Discussion

The developed system successfully demonstrates the working principles of modern vector databases and Retrieval-Augmented Generation systems. The integration of HNSW, KD-Tree, and Brute Force indexing techniques enables comparison between different retrieval approaches within a unified environment. The use of Ollama-based embedding and language models allows the system to operate locally without dependence on external cloud services. Furthermore, the combination of document chunking, semantic retrieval, and AI-generated responses improves the relevance and usefulness of retrieved information. Overall, the project achieved its intended objectives by providing an end-to-end framework for vector storage, semantic search, and AI-assisted question answering.

6. Conclusion

This project successfully developed an AI-Powered Vector Database and Retrieval-Augmented Generation (RAG) system capable of storing, indexing, and retrieving

semantic information from documents. The system integrates multiple vector search techniques, including Brute Force Search, KD-Tree, and Hierarchical Navigable Small World (HNSW), to support efficient similarity search and document retrieval.

The implementation combines a C++ backend, Flutter frontend, and locally hosted Ollama models to create a complete workflow for document upload, embedding generation, semantic retrieval, and AI-assisted question answering. The developed system demonstrates the practical application of vector databases and modern retrieval techniques used in contemporary AI systems.

The project achieved its primary objectives by providing a functional platform for document management, semantic search, and Retrieval-Augmented Generation while allowing users to interact with the system through a user-friendly interface.

REFERENCES

- [1] T. Taipalus, “Vector Database Management Systems: Fundamental Concepts, Use Cases, Challenges, and Future Research Directions,” *Computer Standards & Interfaces*, vol. 92, 2024.
- [2] J. J. Pan, J. Wang, and G. Li, “Survey of Vector Database Management Systems,” *The VLDB Journal*, 2024.
- [3] C. Sharma, “Retrieval-Augmented Generation,” *arXiv preprint arXiv:2506.00054*, 2025.
- [4] M. Klesel and H. F. Wittmann, “Retrieval-Augmented Generation (RAG),” *Business & Information Systems Engineering*, vol. 67, no. 4, pp. 551–561, 2025.
- [5] B. Peng et al., “Graph Retrieval-Augmented Generation: A Survey,” *arXiv:2408.08921*, 2024.

- [6] B. Sarmah et al., "HybridRAG: Integrating Knowledge Graphs and Vector Retrieval-Augmented Generation," arXiv:2408.04948, 2024.
- [7] Z. Jiang et al., "Active Retrieval Augmented Generation," arXiv:2305.06983, 2023.
- [8] Ollama Documentation, "Embeddings for Semantic Search and RAG," 2025.
- [9] Ollama Documentation, "Ollama Official Documentation," Available: <https://docs.ollama.com>
- [10] Ollama Documentation, "Embeddings," Available: <https://docs.ollama.com/capabilities/embeddings>
- [11] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824-836, 2020.
- [12] Flutter Documentation, "Flutter Official Documentation," Available: <https://docs.flutter.dev>
- [13] cpp-http lib Documentation, "cpp-http lib: C++ HTTP/HTTPS Library," Available: <https://github.com/yhirose/cpp-http lib>
- [14] Meta AI, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.
- [15] J. L. Bentley, "Multidimensional Binary Search Trees Used for Associative Searching," Communications of the ACM, vol. 18, no. 9, pp. 509-517, 1975.